



MANUAL TEST GUIDE · MODULE 09 OF 11

Operations Execution · Control Tower Functional-Flow Test Walkthroughs

Scenario-driven QA guide for the **logistics.operations_execution** Control Tower — the reader of physical truth and writer of operational decisions over the mid-flight Globex shipment. Covers activation, plan generation, task & leg execution, milestones, cut-offs, exceptions and the completion gate. Every step traced to its BRD-2 `SOE-*` / `BR-*` rule.

How to use: Work the scenarios top to bottom against a tenant loaded with `--with-demo=all`. Each step gives a precondition, an action, the expected result, and a ✓ **checkpoint** to tick. Green = happy path; amber/red steps deliberately trip a real guard to confirm it holds.

01 What the Control Tower does & how to read this guide

The **Control Tower** turns a `ready_for_execution` shipment into a live, governed `logistics.execution`. It **reads physical truth** (equipment movements, gate events, cut-offs) and **writes operational decisions** (tasks, legs, exceptions, completion), then projects a customer-facing `execution_status` back onto `logistics.shipment` via `@api.extend_model` (SOE-16) — the shipments source file is never edited.

29

COMMANDS TO EXERCISE

3

WALKTHROUGH SCENARIOS

4

BR-01 ACTIVATION GATE INPUTS

8

SOE-13 EXCEPTION CATEGORIES

The state machines you will observe

The execution carries a governed lifecycle; the shipment carries a **derived** operational projection recomputed by `logistics.execution.derive_status` from execution + leg state. Watch both, plus the leg and cut-off machines.

```
execution : draft → activated → in_execution → completed | cancelled
ship.exec  : not_activated → activated → in_execution → on_hold → completed (SOE-16, derived)
leg        : pending → in_progress → completed | skipped
cut-off    : pending → at_risk → met | missed | waived
```

THE BR-01 ACTIVATION GATE

- Shipment must be `ready_for_execution`
- An **owner** is set
(`ops.execution.require_owner_before_activation`)
- No duplicate** active execution for the shipment
- No unresolved critical exception**
(`ops.execution.block_on_critical_exception`)

AUTO-OPENED EXCEPTIONS (SOE-13)

- cutoff_miss** · critical — on a missed cut-off
- cargo_issue** · high — on pickup/delivery damage
- vendor_issue** · medium — on a provider decline
- Eight categories total; severity ladder
low → medium → high → critical

The scenarios in this guide

Scenario	What it proves	Key commands	Section
A — Activate, plan & tasks	BR-01 gate, deterministic plan generation, task lifecycle & dependency, plus the field-ops bookends (pickup / dispatch / driver / provider)	<code>execution.activate</code> , <code>plan.generate</code> , <code>task.complete</code>	\$02
B — Leg execution & tracking	Sequential multi-leg roll-up (SOE-04), milestone planned-vs-actual, SI/VGM cut-offs, append-only events	<code>leg.start / complete</code> , <code>milestone.record_actual</code> , <code>cutoff.evaluate</code>	\$03
C — Exceptions & completion	Exception open → assign → escalate → resolve, the mandatory completion gate, and approval-gated carry-forward (SOE-17)	<code>exception.case.*</code> , <code>execution.complete</code>	\$04
Guard matrix	Every guard that must refuse — with the real <code>ValueError</code> message	consolidated	\$05

Demo fixtures you start from (loaded by `--with-demo=all`)

One live execution `demo_execution_globex` over shipment `CO-SEA-D-2026-000001` , **in_execution**: a generated plan (4 tasks + 5 milestones), a completed pre-carriage **road** leg (factory → INMUM) and an in-progress main **sea** leg (INMUM → SGSIN), the confirmed factory pickup, the SI cut-off (met) and VGM cut-off (at_risk), and one open **vessel-rollover** exception (transport_delay, medium) worked by the ops manager.

02 Activate, generate the plan & run tasks + field ops

Goal: take a ready shipment through the BR-01 gate into a live execution, confirm the plan is generated deterministically, run the task board, and exercise the field-ops bookends. **Role:** Operations manager (`demo_user_ops_manager`). **Start:** a shipment in `ready_for_execution` with an owner set.

#	Action	Expected result	✓ Checkpoint	Trace
1	Run <code>logistics.execution.activate</code> with <code>shipment_id</code> + <code>owner_uid</code> .	New <code>logistics.execution</code> in activated , <code>activated_at</code> stamped; <code>logistics.execution.activated</code> event fires; shipment <code>execution_status</code> projects activated .	Execution created; status = activated.	SOE-01 BR-01
2	Confirm the plan (auto-generated when <code>ops.execution.auto_generate_plan_on_activate</code> is on), or run <code>logistics.execution.plan.generate</code> .	<code>logistics.execution.plan</code> in generated ; <code>task_count</code> / <code>milestone_count</code> populated from the mode-matched templates. Re-running supersedes the prior plan, never duplicates.	Plan generated; re-gen sets old plan superseded.	SOE-02 BR-02
3	Open the task board — tasks instantiated from <code>logistics.task.template.master</code> , each with owner / due / SLA / priority / dependency.	Demo set present: Confirm Booking (done), Generate Pickup Order (done), Submit SI (in_progress), Customs Docs (not_started).	4 tasks; each carries SLA + priority.	SOE-03 BR-04
4	On a task whose <code>depends_on_id</code> is done , run <code>logistics.execution.task.complete</code> .	Task flips to done ; <code>execution.task.completed</code> event fires.	Dependent task completes cleanly.	SOE-03
5	Run <code>logistics.execution.task.reassign</code> with a new <code>owner_uid</code> .	Task <code>owner_uid</code> updated to the new owner.	Owner changed; status untouched.	SOE-03
6	On the factory pickup (proof attached), run <code>logistics.cargo.pickup.confirm</code> .	Pickup → confirmed ; <code>logistics.cargo.pickup_confirmed</code> event. (A damaged pickup would flip to exception + auto-open a <code>cargo_issue</code> case.)	Pickup confirmed with proof.	SOE-05/06 BR-08
7	Drive the pre-carriage dispatch: <code>ops.dispatch.order.start</code> → <code>.complete</code> ; assign a driver via <code>ops.driver.assignment.assign</code> (<code>dispatch_order_id</code> + <code>driver_partner_id</code>), then <code>.accept</code> → <code>.start</code> .	Dispatch → in_progress → completed (<code>ops.dispatch.completed</code>); driver assignment sequenced (relay-aware) and started.	Dispatch completed; driver started.	SOE-01
8	Confirm the nominated trucker with <code>logistics.execution.provider.confirm</code> .	Provider → confirmed ; <code>logistics.execution.provider_confirmed</code> event. (A decline auto-opens a <code>vendor_issue</code> case; a replace supersedes + audits — BR-11.)	Provider confirmed.	SOE-10 BR-11

✓ Scenario A pass criteria

The shipment activated through a clean BR-01 gate, the plan generated deterministically from templates (and superseded cleanly on re-run), the task board ran with dependency respected, and every field-ops bookend confirmed — all from carried-forward data.

⚠ Duplicate activation is refused

The demo already seeds an active execution for the Globex shipment. Re-running `activate` against it must be rejected with `"BR-01: an execution already exists for this shipment"` — one active execution per shipment (see §05).

03 Leg execution, milestones & cut-offs

Goal: prove the Control Tower drives a multi-leg journey as **one execution with sequenced legs** — a prior leg must complete before the next starts (BR-03) — and that leg progress re-derives the shipment status. Then record milestones and evaluate cut-offs. **Start:** the demo execution with its road pre-carriage leg (seq 1) + sea main leg (seq 2).

Leg 1 pre_carriage (road, factory → INMUM) → Leg 2 main_carriage (sea, INMUM → SGSIN)
 each leg: own sequence · leg_type · transport_mode · provider · planned/actual · status
 a leg entering in_progress/completed re-derives ship.exec → **in_execution** (SOE-16)

#	Action	Expected result	✓ Checkpoint	Trace
1	Read the two legs on the execution.	Seq 1 pre_carriage / road , seq 2 main_carriage / sea — each holds its own mode + provider + planned/actual times.	2 sequenced legs, distinct modes.	SOE-04
2	While leg 1 is still pending, run <code>logistics.execution.leg.start</code> on leg 2 .	Refused: "BR-03: a prior leg must complete before this leg can start" (unless <code>ops.leg.allow_overlapping_legs</code> is set).	Out-of-order start blocked.	SOE-04 BR-03
3	Run <code>logistics.execution.leg.start</code> on leg 1 .	Leg 1 → in_progress ; <code>logistics.execution.leg_started</code> + <code>leg_changed</code> events; shipment <code>execution_status</code> re-derives to in_execution .	Leg 1 running; ship status derived.	SOE-16
4	Run <code>logistics.execution.leg.complete</code> on leg 1, then <code>.leg.start</code> on leg 2.	Leg 1 → completed ; leg 2 now starts (prior is completed). Roll-up honoured.	Sequential roll-up holds forward.	SOE-04 BR-03
5	On the Gated In milestone, run <code>logistics.milestone.record_actual</code> with an on-time <code>actual_at</code> .	Milestone → reached ; <code>logistics.milestone.reached</code> event; planned-vs-actual now visible on the timeline.	Actual stamped; status reached.	SOE-12 BR-06
6	Repeat step 5 with a late actual (<code>actual_at > planned_at</code>) and no delay reason.	Refused: "BR-06: a late milestone requires a delay reason". Supply <code>delay_reason</code> → it succeeds.	Late milestone forces a reason.	SOE-12 BR-06
7	On the SI / VGM cut-offs, run <code>logistics.execution.cutoff.evaluate</code> .	State re-derived by <code>planned_at</code> vs <code>alert_threshold_hours</code> : pending → at_risk → missed . A fresh miss auto-opens a critical <code>cutoff_miss</code> exception (<code>ops.cutoff.auto_exception_on_miss</code>).	VGM shows at_risk; a miss raises a case.	SOE-11 BR-05
8	Close a cut-off: <code>logistics.execution.cutoff.mark_met</code> ; and (manager-gated) <code>logistics.execution.cutoff.waive</code> .	<code>met / waived</code> set; <code>cutoff_waived</code> event on waive. A met/waived cut-off is not re-derived by a later evaluate.	Cut-off closed; waive audited.	SOE-11

✓ Scenario B pass criteria

Legs executed strictly in sequence, leg progress re-derived the shipment status (SOE-16), milestones captured planned-vs-actual with the late-reason guard enforced, and cut-offs evaluated with a miss auto-raising a critical exception.

⚠ Cargo events are append-only

Capture a gate event with `logistics.cargo.handover.event.capture` ; then try to edit its `occurred_at` / `event_type` . The `pre.logistics.cargo.handover.event.update` hook vetoes it — only `active` may change. A late event without a reason is refused (BR-06) just like a milestone.

04 Exceptions, the completion gate & carry-forward

Goal: work the vessel-rollover exception through its full lifecycle, then prove the mandatory completion gate refuses an incomplete execution — and that a manager can push it through with an **approval-gated carry-forward** for non-blocking unmet items (SOE-17/18).

Start: the demo execution with its open `transport_delay` exception.

#	Action	Expected result	✓ Checkpoint	Trace
1	Open a case with <code>ops.exception.case.open</code> (payload <code>category</code> or a <code>code_id</code> that resolves one).	Case in open , one of the 8 SOE-13 categories, severity + <code>escalation_chain</code> seeded; <code>ops.exception.opened</code> event. Demo: vessel rollover, <code>transport_delay</code> , medium.	Case opened with a category.	SOE-13 BR-07
2	Run <code>ops.exception.case.assign</code> with an <code>owner_uid</code> .	Owner set; status moves open → investigating .	Assigned; now investigating.	SOE-13
3	Run <code>ops.exception.case.escalate</code> .	<code>ops.exception.escalated</code> event fires carrying the <code>escalation_chain</code> (e.g. <code>dispatcher</code> → <code>ops_manager</code>).	Escalation follows the chain.	SOE-13 BR-07
4	Run <code>ops.exception.case.resolve</code> .	Status → resolved ; <code>ops.exception.resolved</code> event. A resolved case no longer blocks activation or completion.	Case resolved; unblocks gates.	SOE-13
5	With all mandatory tasks done , mandatory milestones reached , no unresolved critical exception, and POD present where delivery is in scope, run <code>logistics.execution.complete</code> .	<code>logistics.execution.completion</code> in completed ; execution → completed ; shipment <code>execution_status</code> → completed ; <code>Execution.Completed</code> event fires for downstream (documentation / billing / tracking).	Execution completes; event fired.	SOE-17/18 BR-09/10
6	Reset an execution with a mandatory task open (or a missing POD) and run <code>logistics.execution.complete</code> with no carry-forward.	Refused: "BR-09/10: execution cannot be completed — ..." listing the live blockers (see below).	Incomplete completion blocked.	SOE-17 BR-09/10
7	Re-run <code>logistics.execution.complete</code> with <code>carry_forward=true</code> + a <code>carry_forward_note</code> (gated by <code>ops.completion.allow_carry_forward</code>).	Completion record in carry_forward , the note retained; execution completes anyway; the blocker list is returned for audit.	Carry-forward completes + records the reason.	SOE-17/18
8	Manual hold / resume: <code>logistics.execution.hold</code> then <code>logistics.execution.resume</code> .	Hold projects ship.exec → on_hold (<code>ops.status.allow_manual_hold</code>); resume re-derives it back from execution + leg state.	Hold/resume toggles derived status.	SOE-16

The completion blockers (BR-09/10)

WHAT BLOCKS COMPLETION

- **mandatory task(s) open** — a mandatory task not `done`
- **mandatory milestone(s) open** — not `reached`
- **unresolved critical exception** — not resolved/closed
- **POD missing where delivery in scope** — no `pod_doc_id`

CARRY-FORWARD SEMANTICS (SOE-17)

- Not a separate command — the `carry_forward` flag on `complete`
- Gated by `ops.completion.allow_carry_forward`
- Completion row stamped `carry_forward`, note kept
- Blockers still returned in the result for audit

05 The guard matrix — every refusal that must hold

Half of testing the Control Tower is confirming it **refuses** bad input. Each row trips a real `ValueError` — the message shown is the exact string raised by the command handler. Reproduce each and tick it.

#	Action	Expected refusal (verbatim message)	Trace
1	<code>execution.activate</code> on a shipment that is not <code>ready_for_execution</code> .	"BR-01: shipment must be 'ready_for_execution' to activate"	BR-01
2	<code>execution.activate</code> with no owner (owner-gate on).	"BR-01: an owner is required before activation"	BR-01
3	<code>execution.activate</code> against a shipment that already has an active execution.	"BR-01: an execution already exists for this shipment"	BR-01
4	<code>execution.activate</code> with an unresolved critical exception on the shipment.	"BR-01: an unresolved critical exception blocks activation"	BR-01
5	<code>execution.task.complete</code> whose dependency task is not done.	"BR-04: cannot complete a task while its dependency is not done"	BR-04
6	<code>execution.leg.start</code> on a leg whose prior leg has not completed.	"BR-03: a prior leg must complete before this leg can start"	BR-03
7	<code>milestone.record_actual</code> with a late actual and no delay reason.	"BR-06: a late milestone requires a delay reason"	BR-06
8	<code>cargo.pickup.confirm</code> with no proof attached (proof-gate on).	"pickup requires proof before it can be confirmed"	BR-08
9	<code>destination.delivery.confirm</code> with no POD where delivery is in scope.	"BR-10: delivery cannot be confirmed without a POD where delivery is in scope"	BR-10
10	Edit a captured <code>cargo.handover.event</code> 's content.	"cargo handover events are append-only — cannot edit [...]"	SOE-08
11	<code>driver.assignment.start</code> before the driver accepted (accept-gate on).	"the driver must accept the assignment before starting"	SOE-01
12	<code>execution.complete</code> with an open mandatory item / missing POD, no carry-forward.	"BR-09/10: execution cannot be completed — ..." (blocker list)	BR-09/10

Config toggles worth flipping during the run

Config key (<code>ir.config</code>)	Default	Effect to verify
<code>ops.execution.auto_generate_plan_on_activate</code>	true	On → the plan is generated automatically when the execution activates.
<code>ops.leg.allow_overlapping_legs</code>	false	On → the BR-03 sequential gate is bypassed; legs may start in parallel.
<code>ops.cutoff.auto_exception_on_miss</code>	true	On → a missed cut-off auto-opens a critical <code>cutoff_miss</code> case.
<code>ops.delivery.require_pod</code>	true	On → delivery cannot confirm without a POD where in scope (BR-10).
<code>ops.completion.allow_carry_forward</code>	true	Gates the manager carry-forward path in §04.
<code>ops.driver.require_accept</code>	false	On → a driver must accept before the run can start.

✓ Full-module pass criteria

All three scenarios green — activation + plan + tasks + field ops (A), sequential multi-leg execution + milestones + cut-offs (B), exception lifecycle + completion + carry-forward (C) — and every guard in this matrix confirmed to refuse with its exact message.

Traceability: SOE-* IDs map to the BRD-2 "Shipment & Operations Execution" document; BR-01...BR-14 to `docs/business_rules.md` (mirrored in `tests/test_business_rules.py`). Downstream effects of `Execution.Completed` (documentation, billing, tracking) are verified in those modules' own guides.